

The portal, feature by feature.

A members' network built on the pattern people already know from LinkedIn — a verified identity, a connection graph, a private inbox, a feed and communities — fitted to one association's actual roster and its actual rules about who gets in.

26

PAGES

58

NEW API FUNCTIONS

25

DATA TABLES

2

WAYS TO SIGN IN

Feature register · Verified against the built application: monorepo typecheck clean, production build green, every route served without a runtime error.

Identity and joining

A directory is worth joining only if the people in it are who they say they are. Every other feature in this document rests on this one, so it is deliberately the strictest part of the system.

HOW A MEMBER GETS IN		
FEATURE	WHAT IT DOES	STATE
Google sign-in	OAuth round trip. Confirms the member's name and email address before any profile exists.	BUILT
LinkedIn sign-in	OAuth round trip. Confirms name, address and current employer — the same fields the directory asks for.	BUILT
Passwords	Removed, not hidden. No password strategy is registered on the backend at all, so there is no unadvertised second way in.	BY DESIGN
Provider health check	Each button is interactive only when the server reports credentials for that provider. With neither set, the card says so plainly instead of showing buttons that fail on click.	BUILT
Roster verification	The provider confirms the email. The association separately checks the batch and roll number against the imported college record.	BUILT
Screening questions	Admin-authored questions asked on the join form, answers attached to the verification request.	BUILT

Consequence, stated plainly: because these two providers are the only doors, a deployment with neither Google nor LinkedIn credentials set has no sign-in at all — including for the association. Setting either one on the Convex deployment switches the card on by itself.

Connections — the friend graph

The request/accept model, with the edge as the unit of truth. Everything private in the portal is gated on an accepted edge rather than on a role.

/NETWORK · 10 BACKEND FUNCTIONS

FEATURE	BEHAVIOUR
Send request	With an optional note. Duplicate and self-requests are refused at the mutation, not just hidden in the UI.
Accept / decline	Only the recipient can answer. Accepting is what opens messaging between the two.
Withdraw	The sender can pull back a pending request before it is answered.
Remove connection	Severs the edge — and with it the ability to message. Enforced on every send, not only at thread creation.
Suggestions	Members from the same batch and department who are not yet connected.
Live pending badge	The header carries an unread count for incoming requests, updated reactively.
Connect from anywhere	A shared action component so a directory profile offers the same connect/message state as the network page.

Private messaging

One-to-one threads between connected members. Real-time by construction: Convex queries are reactive, so a delivered message re-renders the other side's inbox without polling.

/MESSAGES · 6 BACKEND FUNCTIONS

FEATURE	BEHAVIOUR
Open a thread	Addressed by connection id, never by email. Idempotent — clicking "Message" twice lands in the same thread rather than creating a second one.
Send	The connection is re-checked on every send, so removing a connection actually closes the channel. A refusal names its reason.
Inbox	Threads sorted by last activity, each with a counterpart card — name, batch, department, employer, verified mark.
Read receipts	Per-thread read marking, driving an unread count in the site header.
Live delivery	No polling loop and no socket code: the reactive query is the transport.

The gate is the edge. There is no way to open a conversation with a member you are not connected to, and no way to keep one open after the connection is removed. That is the LinkedIn rule, enforced server-side.

Feed and posts

One post component serves both the association-wide feed and every community feed, so the two never drift apart.

/FEED · 12 BACKEND FUNCTIONS

FEATURE	BEHAVIOUR
Post, edit, delete	Authored by any verified member. Edits are permitted to the author; deletion to the author or a moderator.
Likes	Toggle, one per member per post, counted from the join table rather than a mutable tally.
Comments	Threaded under each post, with per-comment deletion rights.
Polls	A post can carry options and collect one vote per member.
Moderation	Hide a post without destroying it, so a decision can be reversed.
Two scopes	The general feed every member sees, and a feed scoped to a single community.

Communities

Groups with their own membership, their own feed, their own moderators and their own entry questions.

/COMMUNITIES AND /COMMUNITIES/[SLUG] · 11 BACKEND FUNCTIONS

FEATURE	BEHAVIOUR
Create	Name, slug, description, and a choice of join mode.
Two join modes	Open — join takes effect immediately. Approval — the request enters a queue a moderator answers.
Screening questions	A community can author its own questions, asked at the point of requesting to join, answers shown to the reviewer.
Approval queue	Per-community, with the applicant's answers attached to each row.
Roles	Members promoted to moderator; moderators manage the queue and the community's feed.
Leave	Self-service, with the last-owner case handled rather than left to fail.
Moderation badge	The header shows a live count of everything waiting on the signed-in member across their communities.

The student roster

Built against the association's real spreadsheets — two files, 234 rows, one identical 16-column format — rather than an invented schema.

THE 16 COLUMNS, AS THEY APPEAR IN THE ASSOCIATION'S FILES

#	COLUMN	REQUIRED	NOTE
1	Sl.No	YES	—
2	First Name	YES	Marked mandatory in the source file
3	Middle Name	YES	0 of 234 rows currently filled
4	Last Name	YES	Marked mandatory
5	Email (Personal)	YES	3 malformed addresses found in the source
6	Secondary Email ID	YES	0 of 234 rows currently filled
7	Phone Number	YES	6 cells contain two numbers
8	Secondary Phone No	YES	—
9	Enrollment Number	YES	The key the join form verifies against
10	Degree	YES	—
11	Department	YES	Normalised on import
12	Permanent Address	YES	Filled on all 234 rows
13	Designation	YES	12 of 234 rows filled
14	Company	YES	32 of 234 rows filled
15	Year of Passing	YES	Range-checked
16	Year of Joining	YES	Range-checked against passing year

Flagging this again, as promised. You chose to make all 16 columns required, including the four the source files leave almost entirely blank. Held to that rule, **all 234 current rows are rejected** until those cells are filled. The importer names every blank cell per row so the association knows exactly what to complete — but nothing imports before that work is done. Relaxing the four sparse columns to optional is a one-line change if you decide differently.

What the importer does

- Parses the workbook in the browser; validates every row on the server.
- Validates per row and reports failures rather than accepting silently — 18 validator checks, tested against the real malformed emails and double-entered phone numbers in your files.
- Records each run as a batch, with a full import history.
- **Undo** — a batch can be rolled back in full.
- Feeds `/students`, the searchable roster, and the verification check on the join form.

The admin panel

An operations console rather than a dashboard of cards — dense tables, every figure live off the database. Four roles: alumni, entrepreneurs, admins and guests.

/ADMIN · WHAT THE CONSOLE COVERS

SECTION	WHAT AN ADMIN CAN DO
Verification queue	Every applicant waiting, with their batch, roll number and screening answers, ready to approve or refuse.
Roster import	Upload the college workbook, see the per-row validation report, commit a batch or undo a previous one.
Verification questions	Author, edit, reorder and retire the questions every applicant is asked.
Membership analytics	Live counts by batch and by department, plus directory totals.
Venture moderation	The RACE entrepreneurship queue — approve, reject, and stage each venture.
Giving ledger	Campaign totals and donations, to the rupee.
Events and mailer	Event list with RSVP counts, and the mailer integration's real status.
Careers and stories	Job postings and published stories, listed for review.
Role grants	Explicit grants, with the source of each member's role shown rather than inferred.

The console reports what is *not* wired as honestly as what is. Push notifications, for instance, are shown as unavailable with the reason given — no subscription table exists to hold a device token — rather than rendered as a switch that does nothing.

The rest of the portal

CARRIED OVER AND KEPT CURRENT

ROUTE	PURPOSE
/directory	The searchable member directory, with individual profiles that now carry connect and message actions.
/students	Roster search over the imported college record.
/dashboard	A member's own landing page — pending requests first.
/profile	The member's own record, editable.
/events · /careers · /mentorship	Events with RSVPs, the job board, and mentorship requests.
/race	The entrepreneurship zone — venture profiles, moderated.
/giving · /stories · /gallery · /newsletters	Campaigns, published stories, albums and newsletters.

How this was verified

CHECKS RUN AGAINST THE BUILT APPLICATION

CHECK	RESULT
Monorepo typecheck	Clean across all packages.
Production build	Green. All 26 pages compiled.
Route smoke test	All 20 primary routes return 200 with no runtime-error markers.
Dynamic routes	Six not-found cases handled without a crash.
Password audit	No password field renders on any route; one social sign-in call exists in the entire web app.
Validator tests	18 checks pass against the real malformed rows in the association's files.

One thing is not done. The Expo mobile app under `apps/native` is still the untouched starter scaffold. Its forms call the password endpoints that the backend no longer registers, so they fail — the files carry a header saying exactly that rather than pretending otherwise. Porting it means adding the OAuth round trip for native and registering a mobile redirect with both providers.